Agent-Controlled Production-Planning Scores

Reusable and versioned mathematical schedule-cost graphs for structured agent priorities and an independently verified small manufacturing job shop.



Concept illustration; not a computed research result. The same 2400 x 1200-pixel source is used in the GitHub README.

Executed research outcome

52 actual paceval graphs | 32 complete policy/model scenarios | 643 multi-graph batches | 3,095,426 executed graph outputs. Every winning schedule and its full cost have been independently verified.

Scope: four synthetic jobs, three machines and two mathematical objective versions. Agent requests are structured fixtures, not live LLM calls. No physical production deployment, real-energy measurements or industrial cost advantage is claimed.

1. Industrial use case and state of the art

Discrete-part manufacturing relies on production planners to route orders through machines while meeting delivery commitments, managing setup changes and using limited equipment time. Each job consists of ordered operations. A machine can process only one operation at a time, and switching product family may introduce a setup interval before the next operation. Managers may prioritize throughput, due-date performance, changeover reduction or a modeled electricity proxy.

A practical planning decision

Consider a hypothetical small manufacturing workshop preparing four orders on three machines. The delivery planner wants all orders finished on time; the maintenance or shop-floor planner wants fewer changeovers. They should not change the feasibility rules merely because their priorities differ. An upstream agent can communicate its priorities through a bounded parameter request, while a deterministic scheduler enumerates feasible plans and paceval evaluates the identical mathematical formula for each plan.

Established solution methods and recent research

Constraint programming, mixed-integer optimization and metaheuristics are established scheduling tools. Google OR-Tools publishes a CP-SAT example with job-precedence and no-overlap constraints [1]. In 2025, Wang et al. investigated multi-agent LLM-based optimization frameworks [2]. Hu et al. (2026) showed that language-driven scheduling reconfiguration requires structural feasibility checking beyond objective-value agreement [3]. A November 2026 survey by Xiang et al. places LLM scheduling agents in the broader modeling-to-solving landscape [4].

The ISA-95 / IEC 62264 framework concerns manufacturing operations integration [6]; ISO 22400 defines manufacturing KPI concepts and terminology [5]. These standards do not validate this synthetic cost model. The present research tests the mathematical scoring layer that might serve a larger production-control architecture, not a substitute for a production scheduling system or safety review.

Research boundary: the small exhaustive scheduler is the combinatorial controller, paceval performs the important stateless mathematical cost and margin functions, and the independent verifier checks both numerical end results and all selected machine allocations.

2. Research architecture and scope

The experiment controls input routes, processing durations, due dates, family-dependent changeovers, model revisions and four agent-supplied priority vectors. The prototype accepts a strictly validated CSV of policies. The C++ host detects cyclic machine orders, builds earliest-start feasible schedules, checks precedence and machine exclusivity, and selects the lowest paceval-computed objective for each selected policy/version combination.

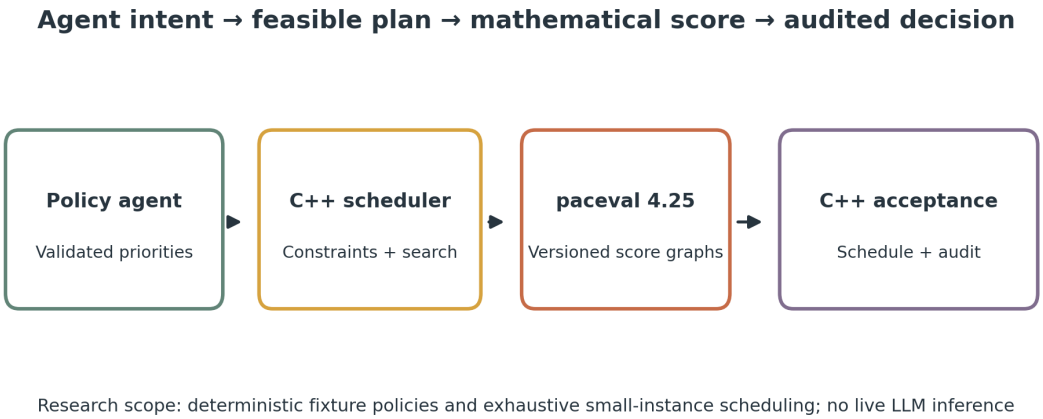


Figure 1. Research system architecture. The agent input is a deterministic fixture; no live language-model inference or tool transport was run.

Two separate questions, two independent checks

Scheduling: Are the selected start and end times globally optimal among the enumerated machine-order assignments for the given scenario and objective? An independent Python enumerator repeats every machine order and checks the published winner. Mathematical execution: Are the reusable original paceval expressions and their complete score values independently reproducible? A hand-written 80-decimal mpmath reference and native C++ checks verify the executed results.

The underlying job shop contains four fixed job routes with three operations per job. For each setup regime, there are 24 cubed possible machine orders, of which 1,860 satisfy the acyclic precedence/resource-order structure. This finite exact search does not establish scalability to real flexible job shops or large factories.

3. Mathematical model and assumptions

Let operation o start at s(o), last p(o) hours and finish at e(o). For each selected machine order, the scheduler enforces both job precedence and machine predecessor constraints. Machine changeover times are symmetric binary-family fixtures; actual tooling and labor setups are not modeled.

$$e_o = s_o + p_o, \qquad s_v \geq e_u + \sigma_{uv}$$

$$\sigma_{uv} = \sigma_0 (f_u - f_v)^2, \quad f \in \{0, 1\}$$

Job tardiness is the positive part of lateness. The signed due-date margin, priority-weighted tardiness and its convex square are real paceval expressions, not native substitutes.

$$T_j = \max(0, C_j - d_j), \quad W_T = \sum_{j=0}^{3} q_j T_j$$

A synthetic electricity surrogate is calculated from fixed machine power coefficients and operation durations; an explicit setup-energy surcharge is added. Normalization denominators are research assumptions, not accounting exchange rates.

$$J_1 = w_M \frac{M}{30} + w_T \frac{W_T}{25} + w_S \frac{S + 0.5 N_S}{20} + w_E \frac{E + 0.25 S}{60}$$

$$J_2 = J_1 + 0.15 w_T \frac{\sum_j q_j T_j^2}{250} + 0.06 \frac{I}{60} + 0.04 \left(\frac{T_{\max}}{15} \right)^2$$

M is makespan, S is setup hours, N(S) the family-change count, I the three-machine idle-time sum, E the synthetic processing energy, and T(max) the largest positive job delay. Weights sum to one. Both graph versions are separately registered and executed, with identical scenario inputs.

4. What the original paceval API actually does

The original paceval 4.25 SDK is initialized, 52 full mathematical expressions are registered and their handles reused. Multi-graph batches evaluate many independent feasible schedules and a separate policy-weight surface. Short scalar calls test graph semantics. All mathematical functions listed as paceval-owned in the register were actually evaluated on meaningful schedule inputs.

Industrial mathematical function	Actual execution	Graph / version
Family-change setup time	paceval	sequence_changeover_hours
Per-job lateness, early completion and weighted delay	paceval	lateness_job*, tardiness_job*, weighted_tardiness_job*
Full priority-weighted delay and squared late delivery	paceval	weighted_tardiness_total, weighted_tardiness_squared
Makespan, setup and synthetic energy score terms	paceval	*_component, energy_total, setup_units
Full versioned objective with time/delivery risk	paceval	policy_objective_v1 and _v2
Due margins, score budget, horizon and sensitivity	paceval	*_margin, d_objective_d_*
Machine order enumeration, discrete precedence and ranking	C++ host	Combinatorial state/decision control
Independent schedule and high precision reconstruction	Python	Verification only; separate code paths

The C++ host carries the structural scheduler and validated policy requests, rather than approximating a combinatorial solver with score expressions. It does not silently execute composite policy objectives in native code. C++ native cost calculations are independent validation only.

Audit trail: results/formula_registry.csv lists original expression strings, results/math_execution_inventory.csv maps every important function to its actual owner, and results/graph_probe_values.csv contains original SDK numerical output for all 32 scenarios.

5. Executed deterministic planning scenarios

Four distinct agent priorities, two family changeover-time regimes, nominal versus rush due dates and two formula versions yield 32 actually executed scenarios. Each deterministic policy is loaded from a validated allowlisted CSV. The host searches every possible machine order and selects the best graph-scored feasible schedule. The synthetic electricity values are not measured production consumption.

Same shop, alternative policy priorities (2-hour setup, nominal due dates, v2)

Agent policy	Makespan	Weighted delay	Setup	Electricity
Delivery-focused	17 h	0 weighted h	10 h	46.2 kWh
Changeover-focused	20 h	6.2 weighted h	6 h	45.2 kWh

The delivery-oriented policy uses a larger tardiness weight, while the changeover-oriented policy trades some lateness for fewer setup hours. Their published objective values are 0.2631 and 0.4791 respectively, but because their weights differ they are not directly comparable as absolute global production quality.

Controlled additional workload: rush due dates

For the 2-hour changeover rush-case under the delivery policy and v2, the chosen plan has a 17-hour makespan, weighted tardiness 12.1 and a computed score of 0.5700. Under that policy's illustrative budget the C++ decision stage labels the result REJECT. This is not an unsafe-equipment determination, but the outcome of a declared numerical planning rule.

The synthetic decision rule produces 19 ACCEPT, 5 REVIEW and 8 REJECT statuses across the 32 cases. The host uses the signed budget margin returned by an original paceval graph. These are internal workflow labels, not physical production authorizations.

6. Executed verification and reproducibility

The C++ application performs explicit independent native checks of representative mathematical components including the full score, the quadratic tardiness risk and signed margins. The Python independent schedule checker reconstructs all machine-order assignments for each setup level and verifies the actual winning schedule under each formula version and policy. The separate 80-digit reference re-derives all 52 mathematical graph quantities for each recorded case, using high-precision input values based on the original binary64 inputs.

Verified artifact	Observed result
Original persistent paceval graphs actually executed	52
Original multi-graph batch calls	643
Original graph outputs returned	3095426
Complete independently verified scenario winners	32
Independent operation-interval checks	768
Independent 80-decimal graph checks	1,664
Native C++ sampled component/full-score checks	18144
Production policy input whitelisting	Accept valid; reject malformed policies

Conditions that are not certified

These tests establish numerical consistency at evaluated points and exhaustive optimization of one small fixed-routing job shop. They do not prove performance scalability, physical machine compatibility, industrial economic benefit, safe autonomous control, LLM inference robustness or a global floating-point error bound. No Trusted Interval Computation (TINC) is used.

The complete reproduction path, external original paceval library requirement, CMake build, CTest and regeneration scripts are recorded in README.md. Numerical comparison details remain in machine-readable audits, not in publication plots or in benchmark-versus-native performance claims.

7.1 Production schedule and scoring policies

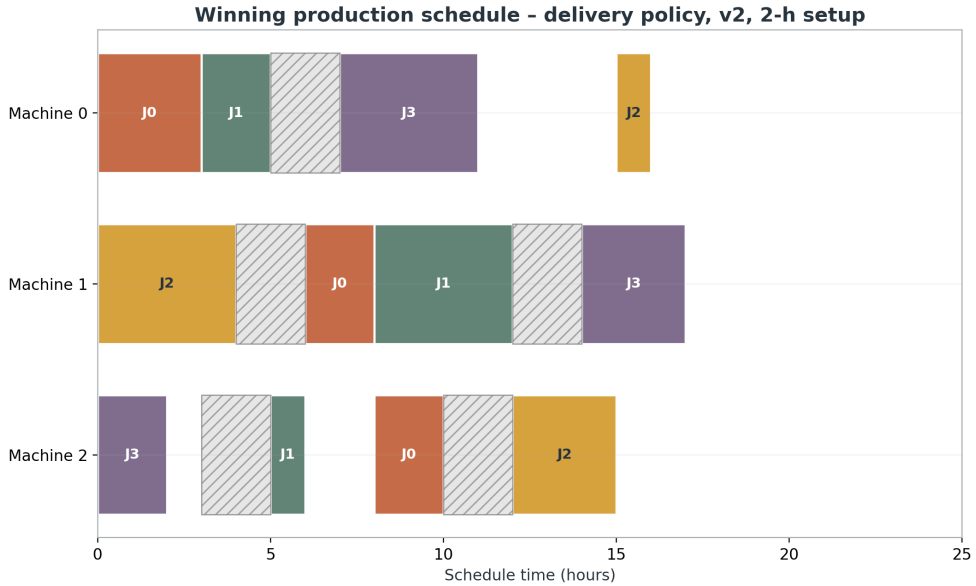


Figure 2. Recorded winning machine-by-machine intervals for the synthetic delivery policy with two-hour family changeovers.

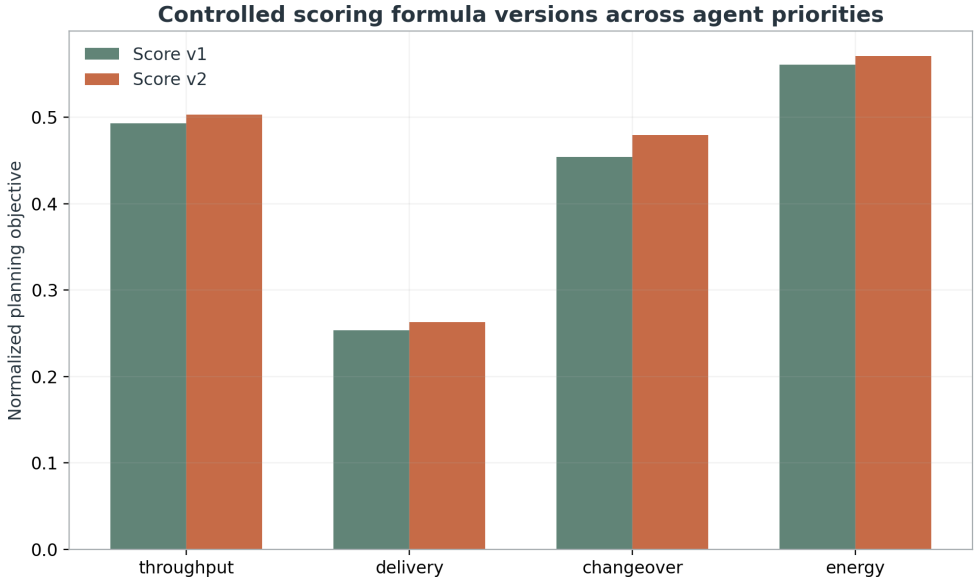


Figure 3. Two persistent score formula versions, separately evaluated by paceval for each declared policy.

7.2 Candidate space and controlled formula changes

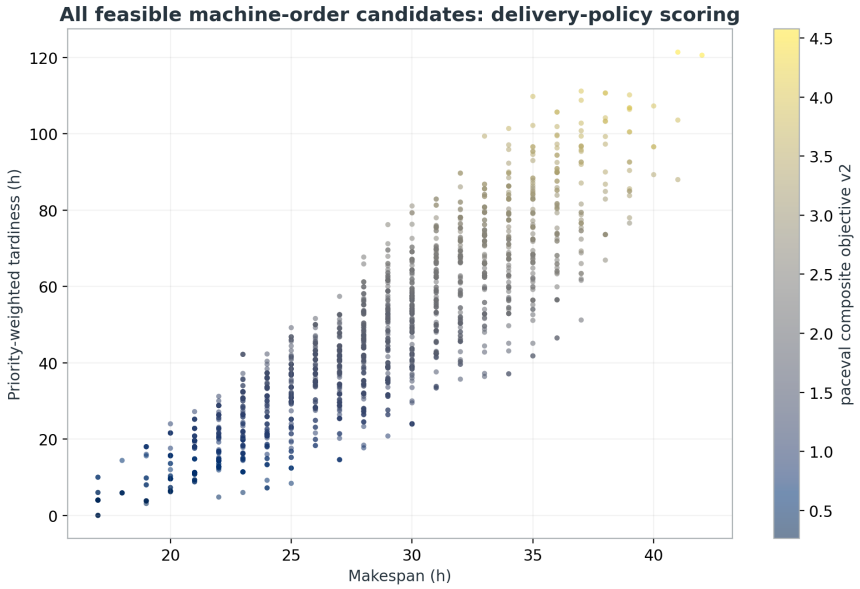


Figure 4. All 1,860 acyclic machine-order candidates with their actual paceval schedule objective and priority-weighted delay.

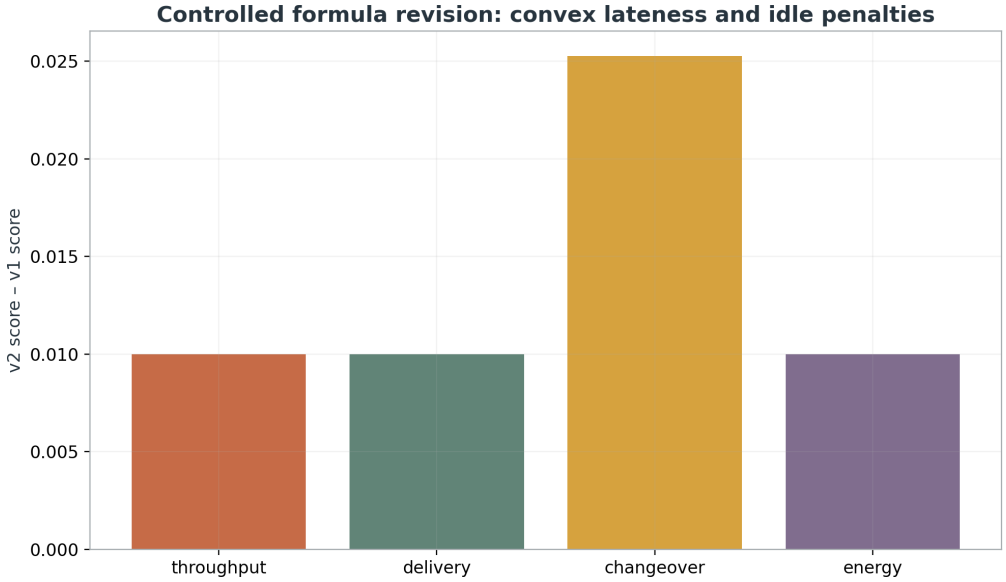


Figure 5. Score change caused by the controlled addition of convex tardiness and idle-time terms.

7.3 Parameter studies and objective trade-offs

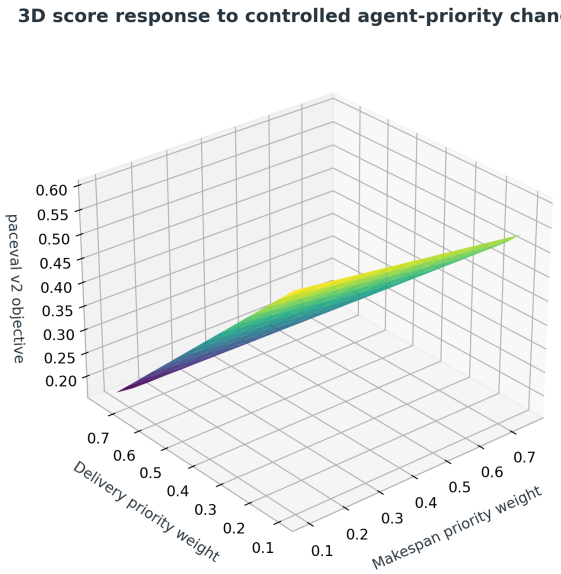


Figure 6. Genuine three-dimensional paceval score response across valid agent-priority weight mixtures at a fixed selected schedule.

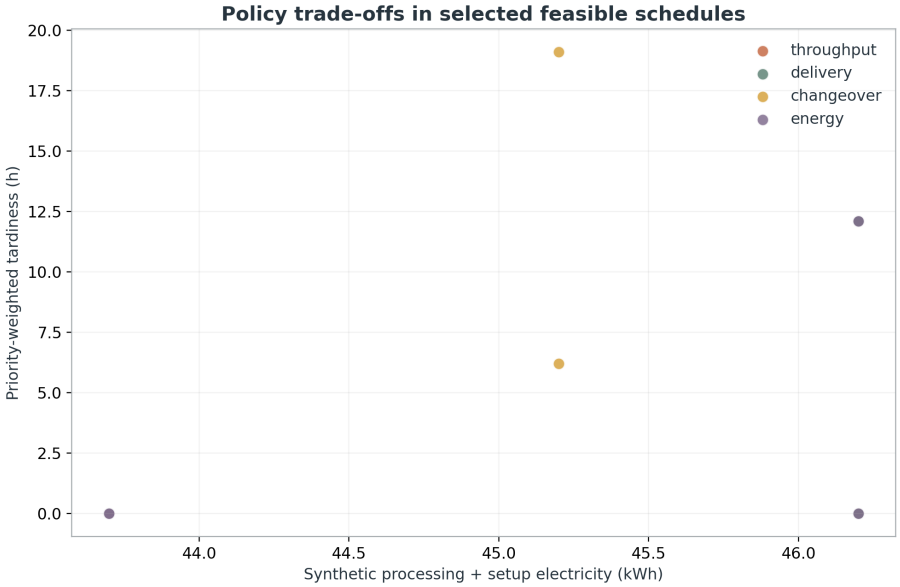


Figure 7. Distinct selected schedule electricity surrogates and tardiness under four documented agent priorities.

7.4 Numerical decision guards and sensitivities

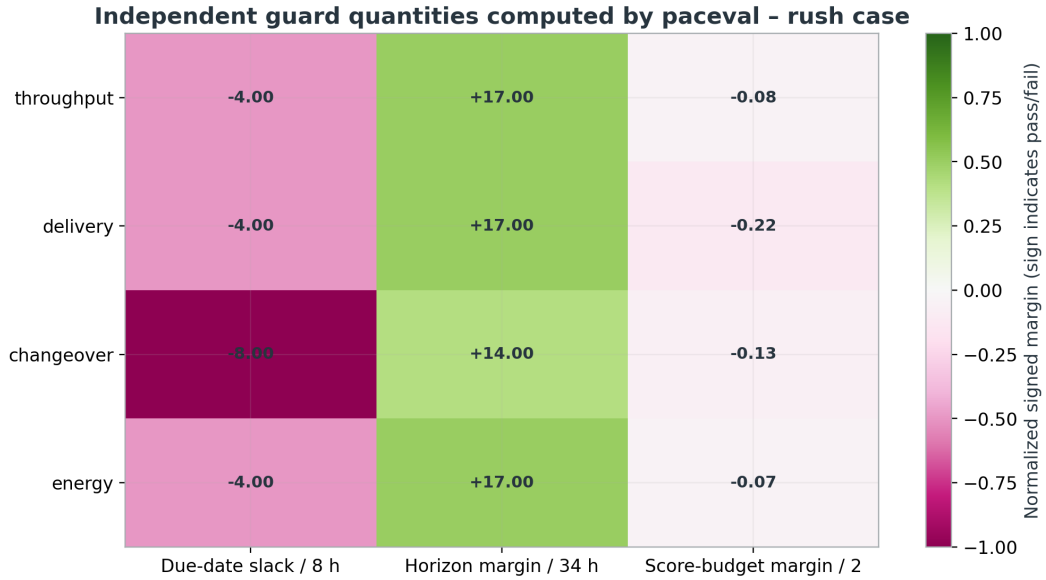


Figure 8. Original paceval signed due-date, horizon and budget margins for the synthetic rush scenario.

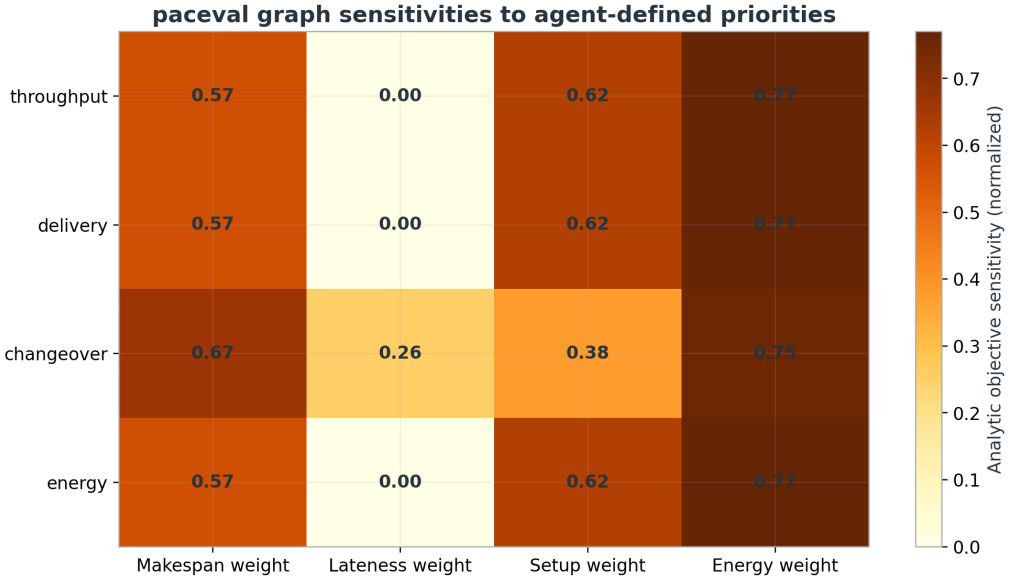


Figure 9. Explicit analytic derivatives of the actual versioned paceval objective with respect to priority weights.

7.5 Meaningful composite outputs and model changes

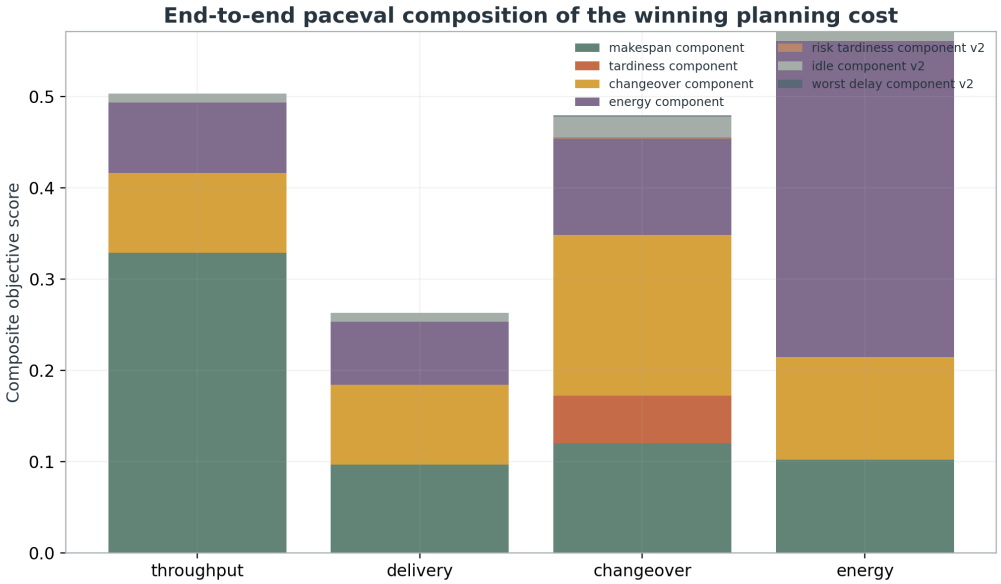


Figure 10. End-to-end component accounting for the actual paceval composite scores; no native substitute supplies the reported components.

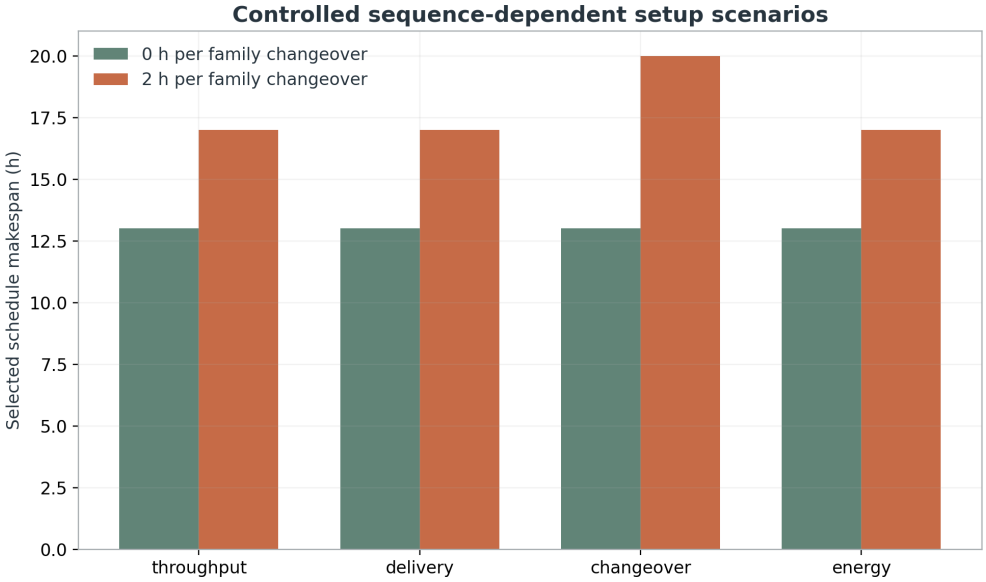


Figure 11. Real selected job-shop makespan changes under two controlled setup-time regimes, chosen by paceval-computed scores.

8. Interpretation, limits and research sources

The successful result demonstrates that a small deterministic agent-to-scheduler-to-paceval workflow can reuse original mathematical execution graphs across independent scheduling candidates, update priority weights without replacing unchanged graph handles, maintain distinct score formula versions and evaluate signed numerical decision margins. It does not show that a language model produced the policy, that paceval solves NP-hard job-shop optimization internally or that any industrial deployment is complete.

Scientific and deployment limitations

All jobs, process durations, two-family changeovers, due dates and power coefficients are synthetic; routing is fixed. The model has no shifts, maintenance, machine downtime, workers, transport resources, sequence-dependent operator crews, capacity buffering or MES data. Real manufacturing requires calibration, constraint coverage review, role authorization, audit of agent instructions and actual system testing. The current agent requests are validated deterministic fixtures. The demonstrated score budgets are not safety margins or regulatory limits.

Selected foundational and recent references

[1] Google OR-Tools, Job shop scheduling with CP-SAT: developers.google.com/optimization/scheduling/job_shop.

[2] Wang, Y., Wang, J., Chu, Z. (2025), Multi-agent large language models as evolutionary optimizers for scheduling optimization. Computers & Industrial Engineering 206, 111197. doi:10.1016/j.cie.2025.111197.

[3] Hu, K. et al. (2026), FJSP-E2E: A process-verified framework for automatic scheduling-model reconfiguration. Journal of Manufacturing Systems. doi:10.1016/j.jmsy.2026.08.009.

[4] Xiang, W. et al. (2026), A survey on large language models for scheduling. Advanced Engineering Informatics 76, 104946. doi:10.1016/j.aei.2026.104946.

[5] ISO 22400-1:2014, manufacturing operations KPIs, confirmed 2025: iso.org/standard/56847.html.

[6] ISA-95 / IEC 62264, Enterprise-Control System Integration: isa.org/standards-and-publications/isa-standards/isa-95-standard.

[7] paceval Mathematical Engine, original C/C++ SDK and API: github.com/paceval/paceval.

Reproduce: original separately supplied Linux paceval runtime, src/CMakeLists.txt, 4 CTest tests, src/high_precision_reference.py, src/verify_results.py, src/make_figures.py, and this PDF-generation script. Source SDK copyright and AGPL notices remain intact.